



QUESTIONS

10 / 10 completed

Integer Array



INTEGER ARRAY

Write a C program to demonstrate pointer arithmetic on an integer array. Display element values using both array indexing and pointer notation. Show address difference between consecutive elements.

Sample Input:

Enter number of elements: 4

Enter elements:

10 20 30 40

Sample Output:

Using Array Indexing:

arr[0] = 10

arr[1] = 20

arr[2] = 30

arr[3] = 40

Using Pointer Notation:

*(ptr+0) = 10

*(ptr+1) = 20

*(ptr+2) = 30

*(ptr+3) = 40

Address Difference:

Address difference between arr[0] and arr[1] = 4 bytes"

PROGRAM EDITOR

C



Run

Save

```
1
2
3 #include <stdio.h>
4
5 int main() {
6     int n, i;
7
8     printf("Enter number of elements: ");
9     scanf("%d", &n);
10
11     int arr[n];
12     int *ptr = arr;
13
14     printf("Enter elements:\n");
15     for (i = 0; i < n; i++) {
16         scanf("%d", &arr[i]);
17     }
18
19     // Using Array Indexing
20     printf("\nUsing Array Indexing:\n");
21     for (i = 0; i < n; i++) {
```

OUTPUT

Enter number of elements: Enter elements:

Using Array Indexing:

Using Pointer Notation:

Address Difference:

Address difference between arr[0] and arr[1] = 4 bytes



QUESTIONS

10 / 10 completed

Factorial



FACTORIAL

"Write a C program to find the factorial of a given number using recursion.

Sample Input: Enter a number- 6

Sample Output: Factorial of 6 is 720

"

PROGRAM EDITOR

C



Run

Save

```
1
2 #include <stdio.h>
3
4 long long factorial(int n) {
5     if (n == 0 || n == 1)
6         return 1;
7     return n * factorial(n - 1);
8 }
9
10 int main() {
11     int n;
12     printf("Enter a number: ");
13     scanf("%d", &n);
14
15     if (n < 0) {
16         printf("Factorial is not defined for negative numbers.\n");
17     } else {
18         printf("Factorial of %d is %lld\n", n, factorial(n));
19     }
20
21     return 0;
```

OUTPUT

```
Enter a number: Factorial of 0 is 1
```

QUESTIONS

10 / 10 completed

Array Rotation

ARRAY ROTATION

"Implement array rotation (left/right) for arrays of size 10 by K positions without using extra space. Use a reversal algorithm. The program must validate K ($0 \leq K < N$), show

original→rotated→reversal steps visually, compute the sum difference, and support multiple rotations.

Sample Input/Output:

Enter 10 numbers: 1 2 3 4 5 6 7 8 9 10

Rotate Left (L)/Right (R): L

By positions: 3

Original: [1 2 3 4 5 6 7 8 9 10]

Step1: Reverse 0-2 → [3 2 1 4 5 6 7 8 9 10]

Step2: Reverse 3-9 → [3 2 1 10 9 8 7 6 5 4]

Step3: Reverse all → [4 5 6 7 8 9 10 1 2 3]

Sum difference: 0"

PROGRAM EDITOR

C

Run

Save

```
1
2
3 #include <stdio.h>
4
5 #define N 10
6
7 void reverse(int arr[], int start, int end) {
8     while (start < end) {
9         int temp = arr[start];
10        arr[start] = arr[end];
11        arr[end] = temp;
12        start++;
13        end--;
14    }
15 }
16
17 void printArray(int arr[], int n) {
18     printf("[");
19     for (int i = 0; i < n; i++) {
20         printf("%d", arr[i]);
21         if (i < n - 1) printf(" ");
```

OUTPUT

```
Enter 10 numbers: Rotate Left (L)/Right (R): By positions: Invalid K! Must be 0 <= K <
10
```


QUESTIONS

10 / 10 completed

Sparse Matrices

SPARSE MATRICES

Represent two sparse matrices (mostly zeros) using arrays of triples [row, col, value]. Read dimensions and non-zero terms for two matrices. Add them, store result in sparse form (sorted by row then col), and display dense matrix form. Max 10x10; ignore zeros.

Sample Input (Matrix 1):

3 3 3

0 1 5

1 0 8

2 2 2

Sample Input (Matrix 2):

3 3 2

0 1 3

2 2 7

Sample Output (Sum Dense):

0 8 0

8 0 0

0 0 9

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int r1, c1, t1, r2, c2, t2;
5
6     // Read Matrix 1
7     scanf("%d %d %d", &r1, &c1, &t1);
8     int m1[10][3];
9     for (int i = 0; i < t1; i++)
10         scanf("%d %d %d", &m1[i][0], &m1[i][1], &m1[i][2]);
11
12     // Read Matrix 2
13     scanf("%d %d %d", &r2, &c2, &t2);
14     int m2[10][3];
15     for (int i = 0; i < t2; i++)
16         scanf("%d %d %d", &m2[i][0], &m2[i][1], &m2[i][2]);
17
18     // Build dense result matrix
19     int result[10][10] = {0};
20
21     for (int i = 0; i < t1; i++)
```

OUTPUT



QUESTIONS

10 / 10 completed

Fibonacci Series



FIBONACCI SERIES

Write a C program to print the first n terms of the Fibonacci series ($n \leq 20$) without using recursion. Start with 0 and 1.

Sample Input/Output:

Enter number of terms: 8

Fibonacci series: 0 1 1 2 3 5 8 13

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i;
5     long long fib[20];
6
7     printf("Enter number of terms: ");
8     scanf("%d", &n);
9
10    fib[0] = 0;
11    fib[1] = 1;
12
13    for (i = 2; i < n; i++) {
14        fib[i] = fib[i-1] + fib[i-2];
15    }
16
17    printf("Fibonacci series: ");
18    for (i = 0; i < n; i++) {
19        printf("%lld ", fib[i]);
20    }
21    printf("\n");
```

OUTPUT

Enter number of terms: Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610

QUESTIONS

10 / 10 completed

Array

ARRAY

Write a C program to find the 5th element in the given array/

Sample Input: a=[10,20,30,40,50,60]

Sample output: 50

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {10, 20, 30, 40, 50, 60};
5
6     printf("%d\n", a[4]);
7
8     return 0;
9 }
```

OUTPUT

50

QUESTIONS

10 / 10 completed

Fibonacci series ($N \leq 50$)**FIBONACCI SERIES ($N \leq 50$)**

Develop a C program to print the first N terms of the Fibonacci series ($N \leq 50$) without using recursion. Use two variables to generate the series iteratively and store the series in an array. The program should also calculate and display the sum of the series and identify the largest term. Input validation is mandatory for N ($1 \leq N \leq 50$).

Sample Input/Output: text

Enter number of terms (1-50): 10

Fibonacci Series: 0 1 1 2 3 5 8 13 21 34

Sum of series: 88

Largest term: 34

Enter number of terms (1-50): 60

Invalid input! Please enter terms between 1 and 50.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {10, 20, 30, 40, 50, 60};
5
6     printf("%d\n", a[4]);
7
8     return 0;
9 }
```

OUTPUT

50

QUESTIONS

10 / 10 completed

Fibonacci series ($N \leq 50$)FIBONACCI SERIES ($N \leq 50$)

Develop a C program to print the first N terms of the Fibonacci series ($N \leq 50$) without using recursion. Use two variables to generate the series iteratively and store the series in an array. The program should also calculate and display the sum of the series and identify the largest term. Input validation is mandatory for N ($1 \leq N \leq 50$).

Sample Input/Output: text

Enter number of terms (1-50): 10

Fibonacci Series: 0 1 1 2 3 5 8 13 21 34

Sum of series: 88

Largest term: 34

Enter number of terms (1-50): 60

Invalid input! Please enter terms between 1 and 50.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int N, i;
5     int fib[50];
6     int a = 0, b = 1;        // two variables to generate Fibonacci
7     long sum = 0;
8     int largest;
9
10    printf("Enter number of terms (1-50): ");
11    if (scanf("%d", &N) != 1) {
12        printf("Invalid input! Please enter terms between 1 and 50.\n");
13        return 0;
14    }
15
16    if (N < 1 || N > 50) {
17        printf("Invalid input! Please enter terms between 1 and 50.\n");
18        return 0;
19    }
20
21    for (i = 0; i < N; i++) {
```

OUTPUT

```
Enter number of terms (1-50): Invalid input! Please enter terms between 1 and 50.
```




QUESTIONS

10 / 10 completed

Unsorted Array

UNSORTED ARRAY

Write a program to find the missing element for the given unsorted array of sequence numbers.

Sample Input: a=[10,9,7,4,3,1,2]

Sample output: 5,6,8

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {10, 9, 7, 4, 3, 1, 2};
5     int n = sizeof(a) / sizeof(a[0]);
6
7     int max = a[0];
8     for (int i = 1; i < n; i++) {
9         if (a[i] > max) max = a[i];
10    }
11
12    int present[max + 1];
13    for (int i = 0; i <= max; i++) present[i] = 0;
14
15    for (int i = 0; i < n; i++) {
16        if (a[i] >= 1 && a[i] <= max) present[a[i]] = 1;
17    }
18
19    int first = 1;
20    for (int i = 1; i <= max; i++) {
21        if (present[i] == 0) {
```

OUTPUT

5,6,8

QUESTIONS

10 / 10 completed

Fib series-iterative & recur

FIB SERIES-ITERATIVE & RECUR

Write a C program to generate Fibonacci series up to N terms using both iterative and recursive methods. The program should display both outputs and compare execution time in milliseconds. Also display total recursive calls made.

Sample Input:

Enter N: 7

Sample Output:

Iterative: 0 1 1 2 3 5 8

Recursive: 0 1 1 2 3 5 8

Recursive Calls: 13

Iterative Time: 0.001 ms

Recursive Time: 0.005 ms

Test Case:

Enter N: 1

Output:

0

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <time.h>
3 #include <stdint.h>
4
5 long long recursive_calls = 0;
6
7 unsigned long long fib_recursive(int n) {
8     recursive_calls++; // count every call
9     if (n == 0) return 0;
10    if (n == 1) return 1;
11    return fib_recursive(n - 1) + fib_recursive(n - 2);
12 }
13
14 int main() {
15     int N;
16     printf("Enter N: ");
17     if (scanf("%d", &N) != 1 || N < 0) {
18         return 0;
19     }
20
21    // ---- Iterative ----
```

OUTPUT

```
Compilation Error:
C:/CCPPComp/CCPP1/bin/./lib/gcc/x86_64-w64-mingw32/10.3.0/./././././x86_64-w64-
mingw32/bin/ld.exe: C:\Windows\TEMP\ccmi8Wv7.o:main.c:(.text+0x155): undefined
reference to `clock_gettime'
C:/CCPPComp/CCPP1/bin/./lib/gcc/x86_64-w64-mingw32/10.3.0/./././././x86_64-w64-
mingw32/bin/ld.exe: C:\Windows\TEMP\ccmi8Wv7.o:main.c:(.text+0x20b): undefined
reference to `clock_gettime'
C:/CCPPComp/CCPP1/bin/./lib/gcc/x86_64-w64-mingw32/10.3.0/./././././x86_64-w64-
mingw32/bin/ld.exe: C:\Windows\TEMP\ccmi8Wv7.o:main.c:(.text+0x26c): undefined
reference to `clock_gettime'
```

QUESTIONS

10 / 10 completed

Structure

STRUCTURE

Write a C program to store student details (Roll No, Name, Marks) using structure and save them into a file. Display highest mark, class average, and total records stored.

Sample Input:

Enter number of students: 2

101 Ravi 75

102 Kiran 85

Sample Output:

Highest Mark: 85 (Kiran)

Class Average: 80.00

2 records stored successfully.

Test Case:

1 student entry → Average = Marks of that student

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2 #include <string.h>
3
4 struct Student {
5     int roll;
6     char name[50];
7     float marks;
8 };
9
10 int main() {
11     int n, i;
12     float sum = 0, highest = -1;
13     struct Student s;
14
15     printf("Enter number of students: ");
16     scanf("%d", &n);
17
18     FILE *fp = fopen("students.dat", "wb");
19     if (fp == NULL) {
20         printf("Error opening file!\n");
21         return 1;
```

OUTPUT

Security Error: Disallowed code pattern detected.